

Guia do Aluno — Lab da Aula 4

Pipeline Olist: GitHub → Airflow → Snowflake

Data Integration e Pipelines · FIAP MBA Data Engineering

Prof. Rafael S Novo Pereira

Apache Airflow 3.2 (Astro Runtime 3.2-5) · Astronomer Astro · Snowflake · Kubernetes

Como usar este guia

Siga as etapas na ordem. Cada uma termina com um ponto de verificação, só avance quando ele estiver OK.

Os quadros  mostram o erro mais comum daquela etapa e como resolver antes de levantar a mão.

Parte 1 (Etapas 1–16): o pipeline rodando — é o desafio Bronze. Parte 2 (Etapas 17–18): duas experiências que valem mais do que parecem. Parte 3: desafios Prata e Ouro.

Antes de começar	3
0.1 — Snowflake ativo	3
0.2 — Conta no GitHub e fork do repositório	3
0.3 — Regras dos trials	3
Parte 1 — O pipeline rodando	4
Etapa 1 — Criar conta no Astro	4
Etapa 2 — Selecionar o uso	4
Etapa 3 — Configurar Organization e Workspace	5
Etapa 4 — Pular o template inicial	6
Etapa 5 — Tela principal do Astro	6
Etapa 6 — Configurar o Deployment	7
Etapa 7 — Aguardar o Deployment ficar HEALTHY	8
Etapa 8 — Verificar o Airflow	9
Etapa 9 — O código no GitHub	9
Etapa 10 — Conectar o GitHub ao Astro IDE	10
Etapa 11 — Deploy do projeto	12
Etapa 12 — Verificar as DAGs no Airflow	13
Etapa 13 — Configurar a Connection com Snowflake	14
Etapa 14 — Executar o pipeline	18
Etapa 15 — Pipeline completo com sucesso	18
Etapa 16 — Verificar os resultados no Snowflake	19
Parte 2 — Duas experiências que valem mais do que parecem	20
Etapa 17 — Backfill: a task não sabe "hoje"	20
Etapa 18 — Idempotência: rodar duas vezes é seguro?	20
Desafios (Extras)	21
🥈 Prata A — Faça um quality check falhar de verdade	21
🥈 Prata B — Dê um calendário à DAG e faça backfill	21
🥈 Prata C — Configure um alerta e prove que disparou	22
🥇 Ouro A — O projeto dbt da Aula 3 orquestrado pelo Airflow	22
🥇 Ouro B — Bronze particionada por data (o pré-requisito real de backfill)	22
Troubleshooting — do erro à solução	24
Checklist de saída	24
O que você acabou de fazer	25

Antes de começar

Três coisas que, se estiverem prontas, poupam quarenta minutos em sala.

0.1 — Snowflake ativo

Você precisa de um trial do Snowflake ativo. O trial dura 30 dias — se o da Aula 2 expirou, crie outro em signup.snowflake.com (3 minutos). Escolha qualquer nuvem/região; anote o e-mail e a senha.

! MFA — leia antes da Etapa 13

Desde 2026 o Snowflake exige autenticação em dois fatores (MFA) para usuários humanos que entram com senha. Isso pode fazer a conexão do Airflow (Etapa 13) falhar com erro de autenticação/MFA.

Se falhar, não perca tempo tentando de novo: siga a Etapa 13B (usuário de serviço com par de chaves). É o jeito certo de fazer em produção — ferramenta nunca usa senha de pessoa.

0.2 — Conta no GitHub e fork do repositório

O código está em github.com/rafsp/Aula3006_MBA. Faça um FORK para a sua conta (botão "Fork" no canto superior direito). Você vai conectar o SEU fork ao Astro — assim consegue editar a DAG nos desafios Prata sem depender de ninguém.

☑ Verificação

Você consegue abrir github.com/SEU-USUARIO/Aula3006_MBA e ver as pastas dags/, datasets/ e os arquivos Dockerfile, requirements.txt, packages.txt.

0.3 — Regras dos trials

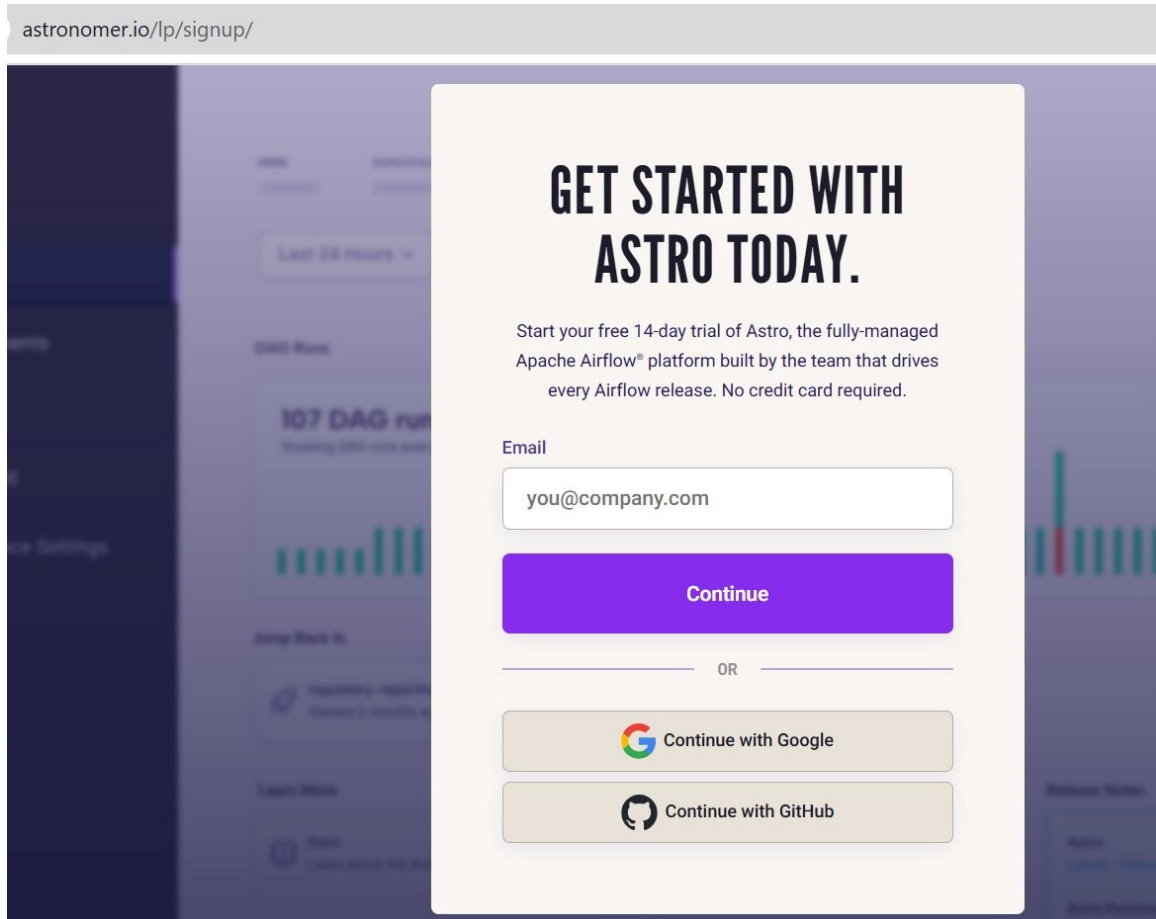
- Astro: 14 dias, US\$ 20 em créditos, sem cartão (valores vistos na tela de signup em ago/2026 — vale o que a tela mostrar). Crie a conta NO DIA da aula, para ela estar viva na entrega dos desafios.
- Snowflake: 30 dias. Anote o Account Identifier no formato ORG-CONTA (ex.: ABCDEFG-XY12345).
- Quando terminar tudo, hiberne ou apague o Deployment do Astro: os créditos são finitos.

Parte 1 — O pipeline rodando

Ao final desta parte você terá um pipeline orquestrado pelo Airflow que baixa 7 CSVs do GitHub, carrega no Snowflake, transforma em Bronze/Silver/Gold e valida a qualidade — sem ninguém tocar no Snowflake. Os screenshots são de uma execução real.

Etapa 1 — Criar conta no Astro

Acesse astronomer.io/lp/signup/ no navegador. Preencha seu e-mail ou use "Continue with Google" / "Continue with GitHub".



astronomer.io/lp/signup/

GET STARTED WITH ASTRO TODAY.

Start your free 14-day trial of Astro, the fully-managed Apache Airflow® platform built by the team that drives every Airflow release. No credit card required.

Email

Continue

OR

 Continue with Google

 Continue with GitHub

O trial é gratuito por 14 dias, sem cartão de crédito, com US\$ 20 em créditos.

Etapa 2 — Selecionar o uso

Na tela "Help us tailor your experience", selecione Professional.

ASTRONOMER

Help us tailor your experience

How do you want to use Astro?



Professional

Build, run, and scale data pipelines



Personal

Experiment with personal projects

Etapa 3 — Configurar Organization e Workspace

Preencha Organization como "Fiap" e Workspace como "Fiap" (ou o nome que preferir). Clique em Continue.

cloud.astronomer.io/start-astro/organization-workspace

ASTRONOMER

Set up your organization

Add your company and team workspace

Organization *

The highest management level in Astro, that can contain multiple Workspaces.

Workspace *

Workspaces contain Airflow Deployments, where permitted users can manage and run their DAGs.

Continue

Etapa 4 — Pular o template inicial

Na tela "How would you like to begin?", clique em "Or skip this and go to your workspace" no final da página. Vamos configurar o projeto manualmente.

ASTRONOMER

How would you like to begin?

Let's create your first project



Upload my own DAGs
Upload your local DAG directory as a .zip file

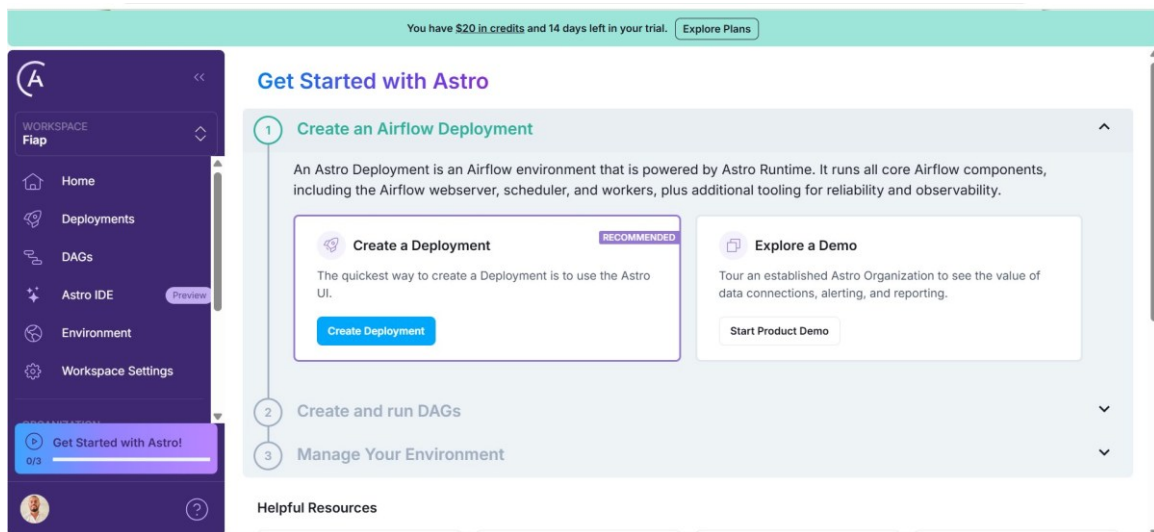


Start with a template
We'll add examples based on the use case you choose

[Or skip this and go to your workspace](#)

Etapa 5 — Tela principal do Astro

Você chega na tela principal. No passo 1, clique no botão verde "Create Deployment".



✓ Verificação

No topo aparece "\$20 in credits and 14 days left in your trial" (ou o valor atual do trial).

Etapa 6 — Configurar o Deployment

6.1 — Nome e template

Nome: pipeline-olist. Template: Development (o mais barato). Provider: Astronomer. Region: westus2.

The screenshot shows the 'New Deployment' page on the Astronomer Cloud console. The URL in the browser is `cloud.astronomer.io/cmn7neu3h00fy01muiac15sjd/deployments/create`. The page title is 'New Deployment'. Under the heading 'CHOOSE A DEPLOYMENT TEMPLATE', there are three cards: 'Development' (Low-cost and secure environment for experimentation and rapid iteration), 'Pre-Production' (Scalable and reliable validation environment for production readiness), and 'Production' (Highly available and reliable environment for business-critical workloads). Each card shows 'Scheduler' and 'Worker Type' options. The 'Development' card is selected. Below the templates, there are sections for 'Cluster' (Standard Cluster or Dedicated Cluster), 'PROVIDER' (Astronomer, AWS, GCP), and 'REGION' (westus2). At the bottom, there are 'Cancel' and 'Create Deployment' buttons.

6.2 — Template Development selecionado

Confirme Development selecionado. Scheduler: Small. Worker Type: A5. Custo indicado: US\$ 0,35/hora + compute.

The screenshot shows the 'New Deployment' page with the 'Development' template selected. The 'Wake Schedules' section is visible, showing a 'Wake Schedule 1' with a 'Switch to cron scheduling' button. The 'Create Deployment' button is visible at the bottom right. The cost is indicated as '\$0.35/hour + Compute (~\$14/month w/ hibernation - 94% savings)'. The 'Compute' is based on the number and type of workers that actually run. See our [detailed pricing](#).

6.3 — Wake Schedule

Configure o Wake Schedule para cobrir o horário em que você vai trabalhar. Schedule Type: Custom. Timezone: Brasilia Time (GMT-03:00). Selecione todos os dias e ajuste FROM/TO. Clique em "Create Deployment".

! Erro comum

Deployment "HIBERNATING" no meio do desafio: o Wake Schedule não cobre o horário. Deployment → Details → Wake Schedule → ajuste. Ele volta em 2–3 min.

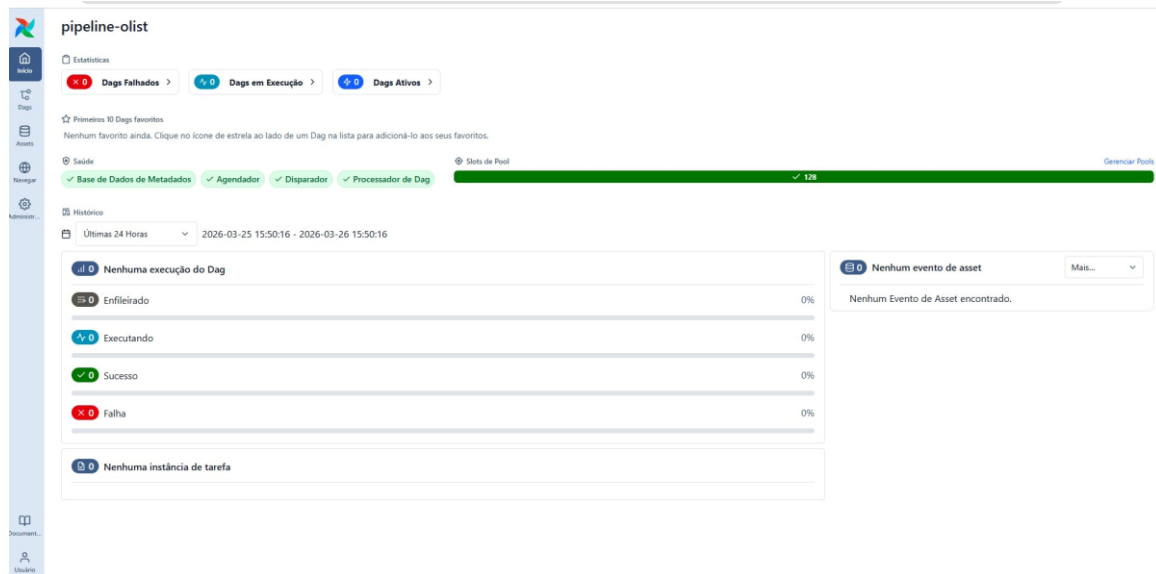
Etapa 7 — Aguardar o Deployment ficar HEALTHY

Aguarde 2–5 minutos. O status muda de "CREATING" para "HEALTHY" e aparece o botão "Open Airflow" no canto superior direito.

Deployments > pipeline-olist DEV						Open Airflow DAGs ...	
ID	BRANCH MAPPING	DOCKER IMAGE	DAG BUNDLE VERSION	UPDATED	CREATED		
2 cmn7...fvos	None - Configure	3.1-14	—	an hour ago by Rafael Pereira	an hour ago by Rafael Pereira		

Etapa 8 — Verificar o Airflow

Clique em "Open Airflow". A UI do Airflow 3 abre mostrando os componentes saudáveis: Base de Dados de Metadados, Agendador (Scheduler), Disparador (Triggerer) e Processador de Dag. Todos verdes.



💡 Conecte com a aula

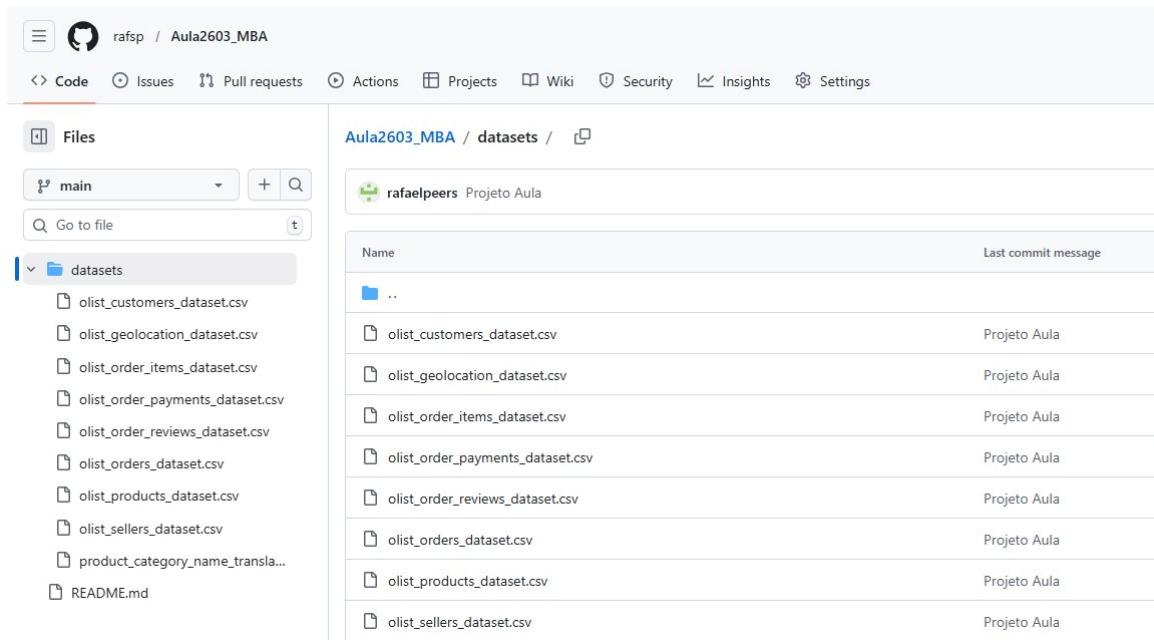
São as peças do slide "Arquitetura do Airflow 3": Metadata DB (o caderno), Scheduler (o chef), Triggerer (quem segura tasks que esperam evento) e DAG Processor (quem lê a receita). Os Workers não aparecem aqui porque nascem como pods só quando uma task roda.

Neste ponto o Airflow está rodando, mas ainda sem nenhuma DAG. Vamos resolver isso nas próximas etapas.

Etapa 9 — O código no GitHub

Abra o seu fork (github.com/rafsp/Aula3006_MBA). O repositório contém:

Caminho	O que é
datasets/	8 CSVs do Olist (a DAG usa 7 — geolocation fica de fora)
dags/pipeline_olist.py	A DAG principal: pipeline_olist_completo (6 tasks)
dags/demo_backfill.py	A demo de backfill vista na aula (Etapa 17)
dags/pipeline_olist_dbt.py	Desafio Ouro: o projeto dbt da Aula 3 orquestrado pelo Airflow (Cosmos)
dags/dbt/olist/	O projeto dbt da Aula 3
setup/01_snowflake_service_user.sql	Plano B da Etapa 13 (usuário de serviço + chave)
Dockerfile · requirements.txt · packages.txt	Receita da imagem Docker: Astro Runtime 3.2-5 (Airflow 3.2), provider do Snowflake, pandas, Cosmos, venv do dbt

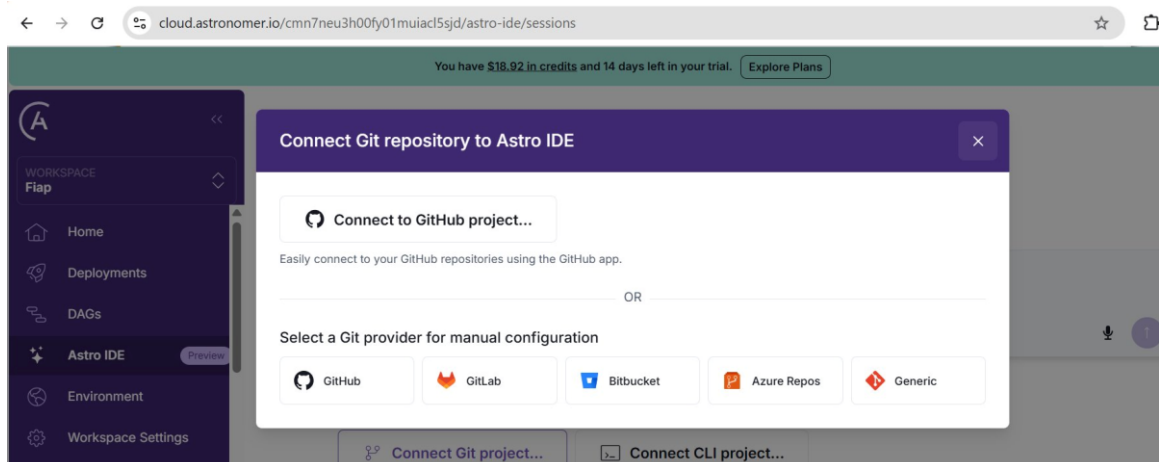


Os três arquivos da raiz são o que o Astro usa para construir a imagem Docker com as dependências certas. Sem eles, a task de Bronze falharia com "No module named snowflake".

Etapa 10 — Conectar o GitHub ao Astro IDE

10.1 — Abrir o Astro IDE

No Astro, clique em "Astro IDE" no menu lateral. Depois em "Connect Git project..."



10.2 — Autorizar o GitHub

Clique em "Connect to GitHub project..." e autorize o acesso ao seu GitHub quando solicitado.

10.3 — Selecionar o repositório

Selecione o SEU fork (Aula3006_MBA). Deixe o campo "Astro Project Path" VAZIO — o projeto está na raiz. Git Branch: main. Clique em Connect.

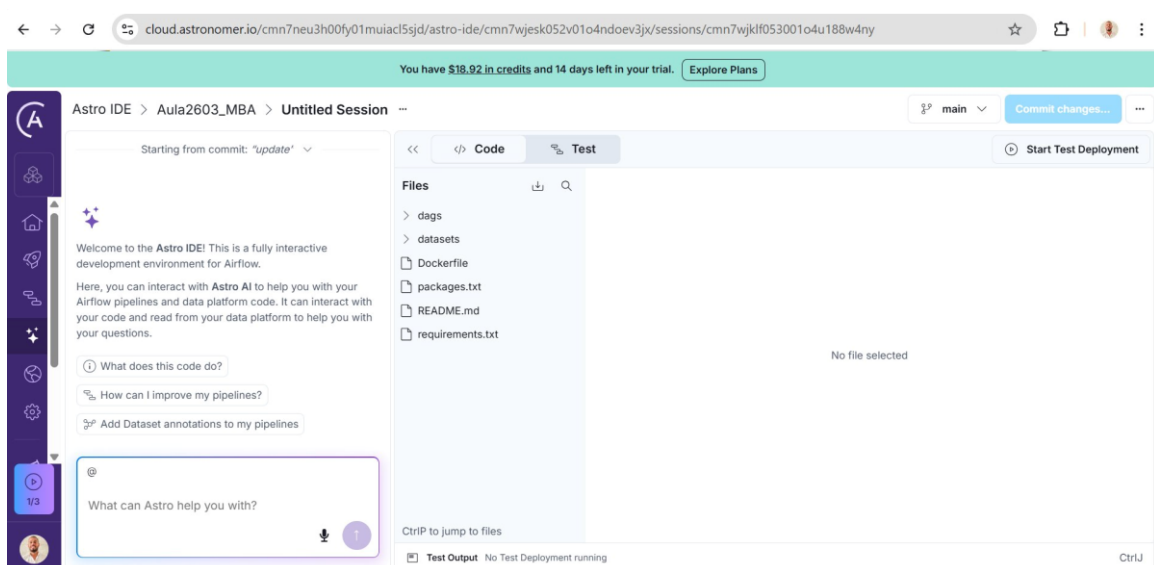
The screenshot shows a modal window titled "Connect GitHub repository to Astro IDE". Inside, there's a section "Select a repository". Under "REPOSITORY *", a dropdown menu shows "Aula2603_MBA". Below it, a note says "Only repositories that the GitHub app is authorized to access will be available. [Manage access in GitHub.](#)". Under "ASTRO PROJECT PATH", a text input field contains "path/to/project-directory", with a note below it: "The location of your Astro project, if not in the repository root." Under "GIT BRANCH", a text input field contains "main", with a note below it: "The default branch to use for this project. If not specified, the default branch of the repository will be used." At the bottom, there are three buttons: "Back", "Cancel", and "Connect".

! Erro comum nº 1 do lab

Preencher "Astro Project Path" com "days" ou "/". Tem que ficar vazio. Se errou: desconecte o projeto no IDE e conecte de novo.

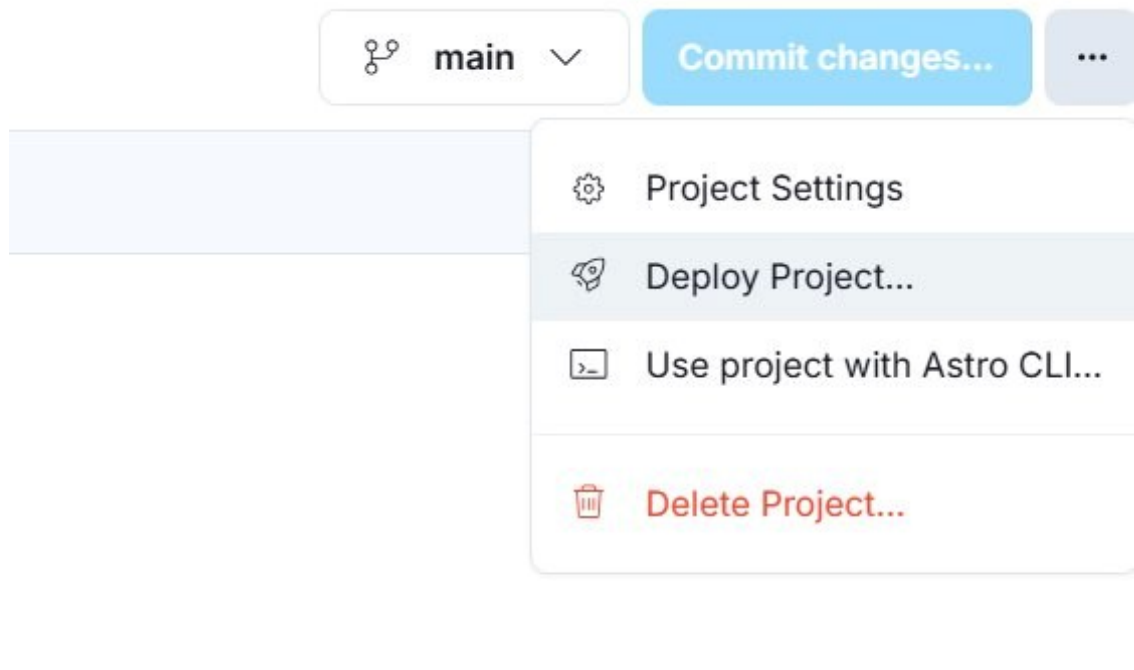
10.4 — IDE conectado

O Astro IDE abre mostrando os arquivos: dags, datasets, setup, Dockerfile, packages.txt, README.md, requirements.txt.

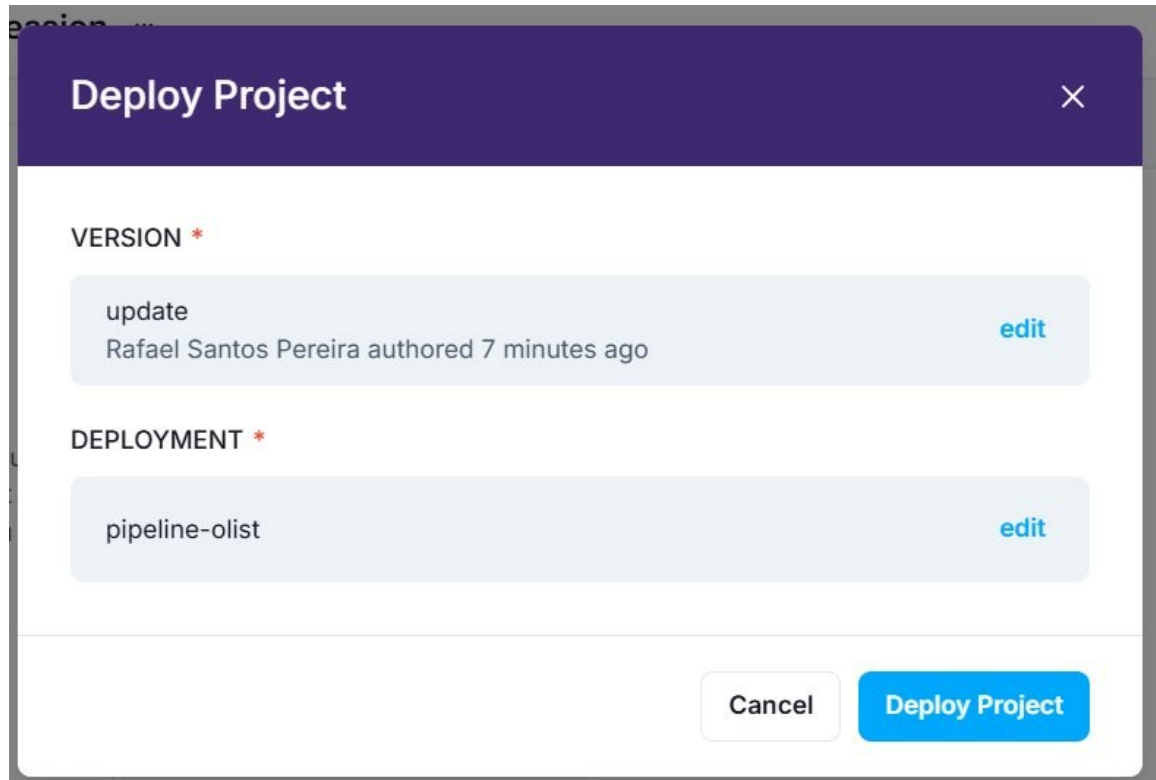


Etapa 11 — Deploy do projeto

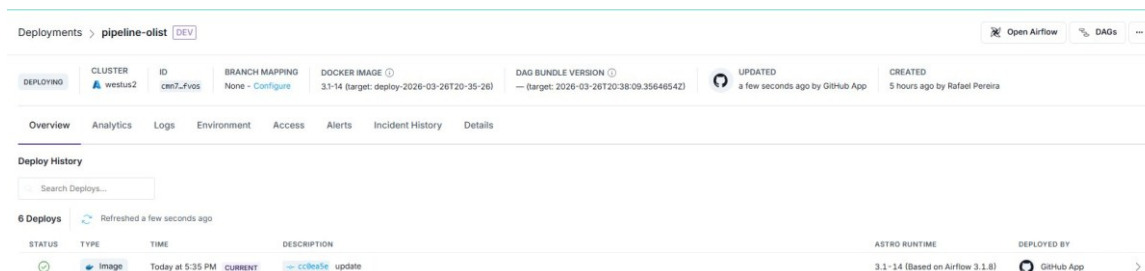
Clique nos três pontinhos "..." no canto superior direito do IDE e selecione "Deploy Project...".



Na janela de confirmação, verifique que o Deployment é "pipeline-olist" e clique em "Deploy Project".



Aguarde 3–5 minutos. O Astro constrói a imagem Docker, instala as dependências e sobe as DAGs. Quando o status do Deployment voltar para HEALTHY, o deploy terminou.



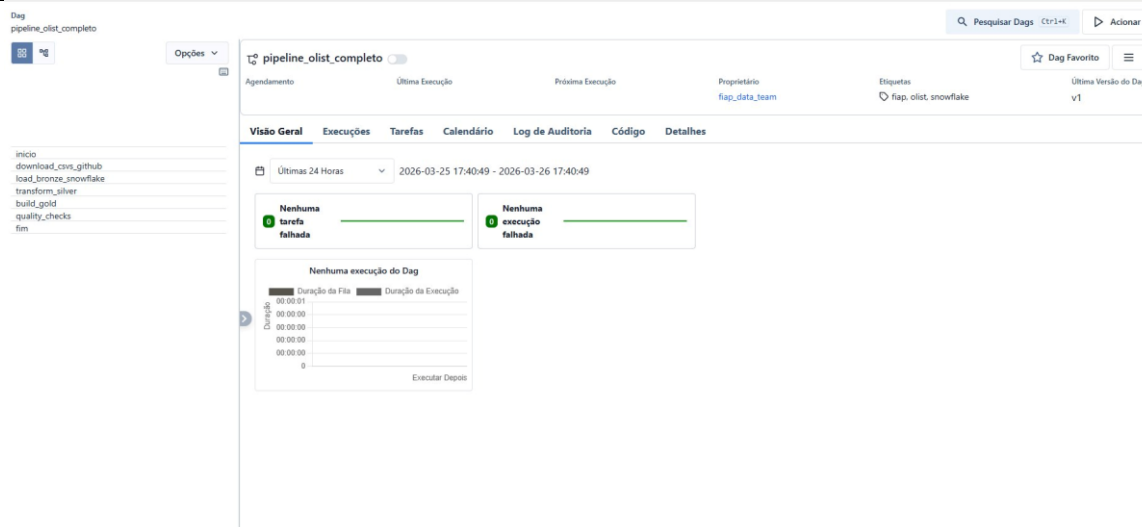
💡 O que aconteceu por baixo

Seu código virou uma imagem Docker (Python + bibliotecas + DAGs). A imagem subiu no cluster Kubernetes do Astro. A partir de agora, cada task que rodar vai nascer como um pod feito dessa imagem. É o slide "Astro: vocês já estão usando Kubernetes".

Etapa 12 — Verificar as DAGs no Airflow

Clique em "Open Airflow". Três DAGs aparecem na lista:

DAG	Para quê	Quando usar
pipeline_olist_completo	O pipeline do lab: inicio → download_and_load_bronze → transform_silver → build_gold → quality_checks → fim	Etapa 14
demo_backfill	A demo da aula: schedule diário + task que lê logical_date	Etapa 17
pipeline_olist_dbt	Desafio Ouro (Cosmos + dbt). NÃO rode agora.	Parte 3



✅ Verificação

As três DAGs estão na lista e nenhuma mostra "Import Error". Se aparecer erro de import, veja a tabela de troubleshooting no fim do guia.

! DAG não aparece?

Espera o Deployment voltar a HEALTHY (3–5 min) e recarregue a página. O DAG Processor só lê os arquivos depois que a imagem nova está no ar.

Etapa 13 — Configurar a Connection com Snowflake

❌ CHECKPOINT 1 — levante a mão antes de clicar em Salvar

Esta é a etapa mais importante. Sem a Connection correta, a task de Bronze fica vermelha (três tentativas) e as seguintes ficam upstream_failed. Confira os quatro campos com o professor antes de salvar.

A senha do Snowflake NUNCA fica no código. Ela fica numa Connection do Airflow, criptografada no Metadata DB. O código só diz "use a conexão snowflake_default".

```
ERROR - Task failed with exception
▼ AirflowNotFoundException: The conn_id 'snowflake_default' isn't defined
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/sdk/execution_time/task_runner.py", linha 1579 em _run_task_and_map_outcome
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/sdk/execution_time/task_runner.py", linha 197 em wrapper
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/sdk/execution_time/task_runner.py", linha 2234 em _execute_task
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/sdk/execution_time/task_runner.py", linha 197 em wrapper
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/sdk/execution_time/task_runner.py", linha 2187 em _run_execute_callable
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/sdk/bases/operator.py", linha 445 em wrapper
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/sdk/bases/decorator.py", linha 398 em execute
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/sdk/bases/operator.py", linha 445 em wrapper
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/providers/standard/operators/python.py", linha 231 em execute
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/providers/standard/operators/python.py", linha 254 em execute_callable
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/sdk/execution_time/callback_runner.py", linha 97 em run
Arquivo "/tmp/airflow/dag_bundles/astro/main/dags/2026-08-27T20:50:41.8621129Z/dags/pipeline_olist.py", linha 75 em download_and_load_bronze
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/providers/snowflake/hooks/snowflake.py", linha 761 em get_conn
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/providers/snowflake/hooks/snowflake.py", linha 435 em _get_conn_params
Arquivo "/usr/local/lib/python3.14/functools.py", linha 1126 em __get__
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/providers/snowflake/hooks/snowflake.py", linha 647 em _get_static_conn_params
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/sdk/bases/hook.py", linha 61 em get_connection
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/sdk/definitions/connection.py", linha 280 em get
Arquivo "/usr/local/lib/python3.14/site-packages/airflow/sdk/execution_time/context.py", linha 230 em _get_connection
```

13.1 — Encontrar suas credenciais do Snowflake

Abra o Snowflake no browser. Clique no seu nome (canto inferior esquerdo) → Account Details. Anote o Account Identifier (formato ORG-CONTA) e o Login name.

Account Details		
Account	Config File	Connectors/Drivers
SQL Commands		
NAME	VALUE	
Account identifier ⓘ	AJQWLVN-XJC53275	
Data sharing account identifier ⓘ	AJQWLVN.XJC53275	
Organization name	AJQWLVN	
Account name	XJC53275	
Account/Server URL	AJQWLVN-XJC53275.snowflakecomputing.com	
Login name ⓘ	RAFAELNOVO	
Role	ACCOUNTADMIN	
Account locator	DHC22010	
Cloud platform	AWS	
Edition	Enterprise	
Learn more		Close

13.2 — Criar a Connection no Airflow

No Airflow, clique na engrenagem (Admin) → Conexões (Connections) → botão "+".
Preencha:

Campo	Valor
ID da Conexão (Connection Id)	snowflake_default
Tipo de Conexão (Connection Type)	Snowflake

13.3 — Campos padrão (Etapa 13A — com senha)

Campo	Valor
login	seu Login name do Snowflake (ex.: MARIASILVA)
password	sua senha do Snowflake
schema	PUBLIC

Adicionar Conexão

ID da Conexão *

snowflake_default

Tipo de Conexão *

Snowflake

Tipo de conexão faltando? Certifique-se de ter instalado o pacote do provider correspondente ao Airflow.

Campos Padrão

Descrição

login

RAFAELNOVO

password

MbaEngenharia@2016

schema

PUBLIC

Campos Extra

Campos Extra JSON

Salvar

13.4 — Campos Extra JSON

Clique em "Campos Extra JSON" para expandir. Cole o JSON abaixo, trocando o account pelo seu Account Identifier:

```
{"account": "SEU-ORG-SUA-CONTA", "warehouse": "COMPUTE_WH",  
  "database": "OLIST_LAB", "role": "ACCOUNTADMIN"}
```

Adicionar Conexão

ID da Conexão *

snowflake_default

Tipo de Conexão *

Snowflake

Tipo de conexão faltando? Certifique-se de ter instalado o pacote do provider correspondente ao Airflow.

Campos Padrão

Campos Extra

Campos Extra JSON

1 {

2 "account": "AJQWLVN-XJC53275",

3 "warehouse": "COMPUTE_WH",

4 "database": "OLIST_LAB",

5 "role": "ACCOUNTADMIN"

6 }

Salvar

! Erro comum nº 2 do lab

account no formato errado. É ORG-CONTA (ex.: ABCDEFG-XY12345). NÃO inclua ".snowflakecomputing.com", "https://" nem a região. Se a URL da sua conta é https://abcdefg-xy12345.snowflakecomputing.com, o account é abcdefg-xy12345.

Clique em Salvar. Depois clique em "Test" (se disponível no seu Deployment) — ou vá direto para a Etapa 14: a task de Bronze é o teste real.

✓ Verificação

A Connection snowflake_default aparece na lista com tipo Snowflake.

13B — Plano B: usuário de serviço com par de chaves (se a Bronze falhar com erro de MFA/autenticação)

Se, na Etapa 14, o log da task download_and_load_bronze mostrar algo como "Multi-factor authentication is required", "MFA", "250001" ou "Incorrect username or password" (com a senha certa), o Snowflake está exigindo MFA do seu usuário humano. A solução — e a prática correta em produção — é um usuário de serviço que se autentica por chave, não por senha. Leva 10 minutos.

Passo 1 — Gerar o par de chaves no seu computador. Abra um terminal (macOS/Linux: Terminal; Windows: Git Bash ou PowerShell com OpenSSL) e rode:

```
openssl genrsa 2048 | openssl pkcs8 -topk8 -inform PEM -out rsa_key.p8 -nocrypt  
openssl rsa -in rsa_key.p8 -pubout -out rsa_key.pub
```


Isso cria `rsa_key.p8` (chave PRIVADA — nunca compartilhe, nunca faça commit) e `rsa_key.pub` (chave pública).

Passo 2 — Registrar a chave pública no Snowflake. Abra `rsa_key.pub` num editor. Copie SOMENTE o miolo (as linhas entre `-----BEGIN PUBLIC KEY-----` e `-----END PUBLIC KEY-----`), junte tudo numa linha. No Snowflake, em Projects → SQL File, abra o arquivo `setup/01_snowflake_service_user.sql` do repositório, cole o miolo no campo `RSA_PUBLIC_KEY` e execute o script inteiro como `ACCOUNTADMIN`.

O script cria: a role `AIRFLOW_ROLE` (com o mínimo necessário sobre `OLIST_LAB` e `COMPUTE_WH`), o usuário `AIRFLOW_SVC` com `TYPE = SERVICE` (não humano → fora da regra de MFA) e registra a chave. No fim, `DESC USER AIRFLOW_SVC` deve mostrar `RSA_PUBLIC_KEY_FP` preenchido.

Passo 3 — Refazer a Connection no Airflow. Edite `snowflake_default` (ou apague e crie de novo):

Campo	Valor
login	<code>AIRFLOW_SVC</code>
password	(deixe VAZIO)
schema	<code>PUBLIC</code>

Extra JSON — cole o CONTEÚDO INTEIRO do `rsa_key.p8` (com as linhas `BEGIN/END`) no campo `private_key_content`, trocando as quebras de linha por `\n`:

```
{ "account": "SEU-ORG-SUA-CONTA", "warehouse": "COMPUTE_WH", "database":  
"OLIST_LAB",  
  "role": "AIRFLOW_ROLE",  
  "private_key_content": "-----BEGIN PRIVATE KEY-----\nMIIEvQIBADANBg...\n...\n-----END PRIVATE KEY-----"} }
```

💡 Dica para montar o JSON sem erro

No terminal: `python3 -c "import json;print(json.dumps(open('rsa_key.p8').read()))"` — isso imprime a chave já escapada com `\n`, pronta para colar depois de `"private_key_content":`.

✅ Verificação

Rode a Etapa 14 de novo. Se a Bronze ficar verde, a Connection por chave está funcionando. Para o desafio Ouro (dbt), adicione a variável de ambiente `SNOWFLAKE_AUTH=keypair` no Deployment (Astro → Deployment → Environment) — a DAG dbt usa isso para saber que deve autenticar por chave.

Volte para a lista de DAGs. Clique em `pipeline_olist_completo`. Clique em "Acionar" (Trigger, o botão play no canto superior direito). Selecione "Execução Única" e confirme.

Acompanhe as tasks na lateral: elas mudam de cor conforme avançam pela jornada (scheduled → queued → running → success). A execução completa leva cerca de 2 minutos — a Bronze é a mais longa (baixa 50 MB e grava 7 tabelas).

Clique no quadrado vermelho → Logs. Leia a ÚLTIMA linha do log. "authentication", "MFA", "250001" → Etapa 13B. "account" / "could not connect" → confira o account no Extra JSON (Etapa 13.4). "No module named" → o deploy não usou o requirements.txt (Etapa 10.3: Project Path tem que estar vazio). Depois de corrigir: clique na task → "Clear" → ela roda de novo (e as seguintes também).

Quando todas as tasks ficarem verdes, o pipeline terminou. Veja a duração de cada task e o total.

Observe: inicio → download_and_load_bronze (59 s) → transform_silver (7 s) → build_gold (6 s) → quality_checks (5 s) → fim. Total ≈ 1 min 49 s. Abra o log de quality_checks: ele imprime as 4 regras com OK e o top 5 de estados por receita.

✓ CHECKPOINT 2 — evidências do desafio Bronze

Tire agora os 3 screenshots (com seu nome de usuário visível no canto da tela): (1) a Grid com as 6 tasks verdes; (2) o log de download_and_load_bronze mostrando as 7 tabelas e o número de linhas; (3) — na Etapa 16 — o resultado do SELECT na Gold.

Etapa 16 — Verificar os resultados no Snowflake

Abra o Snowflake em Projects → SQL File (a antiga "Worksheet") e execute:

```
SELECT * FROM OLIST_LAB.PUBLIC.GOLD_RECEITA_ESTADO ORDER BY RECEITA_TOTAL DESC
LIMIT 5;
```

The screenshot shows the Snowflake SQL interface. At the top, the query is entered: `SELECT * FROM OLIST_LAB.PUBLIC.GOLD_RECEITA_ESTADO ORDER BY RECEITA_TOTAL DESC LIMIT 5;`. Below the query editor, there are buttons for "Add to Chat", "Explain", "Quick edit", and "Format". The results section, titled "Results (just now)", shows a table with 5 rows and 8 columns. The columns are: ESTADO, TOTAL_PEDIDOS, RECEITA_TOTAL, TICKET_MEDIO, DIAS_ENTREGA_MEDIO, PCT_ATRASO, and _ATUALIZADO_EM. The rows represent data for different states: SP, RJ, MG, RS, and PR.

	ESTADO	TOTAL_PEDIDOS	RECEITA_TOTAL	TICKET_MEDIO	DIAS_ENTREGA_MEDIO	PCT_ATRASO	_ATUALIZADO_EM
1	SP	41746	5998226.96	143.69	8.7	5.72	2026-03-26 14:05:50.714 -0700
2	RJ	12852	2144379.69	166.85	15.2	12.95	2026-03-26 14:05:50.714 -0700
3	MG	11635	1872257.26	160.92	11.9	5.48	2026-03-26 14:05:50.714 -0700
4	RS	5466	890898.54	162.99	15.2	6.99	2026-03-26 14:05:50.714 -0700
5	PR	5045	811156.38	160.78	11.9	4.88	2026-03-26 14:05:50.714 -0700

Compare com os valores esperados — eles são reproduzíveis, quem rodar o pipeline vê os mesmos:

ESTADO	TOTAL_PEDIDOS	RECEITA_TOTAL	TICKET_MEDIO	DIAS_ENTREGA_MEDIO	PCT_ATRASO
SP	41746	5998226.96	143.69	8.7	5.72
RJ	12852	2144379.69	166.85	15.2	12.95
MG	11635	1872257.26	160.92	11.9	5.48

Agora as outras tabelas que o pipeline criou:

```
SHOW TABLES IN OLIST_LAB.PUBLIC; -- 7 BRONZE_*, SILVER_PEDIDOS, 2 GOLD_*
SELECT * FROM OLIST_LAB.PUBLIC.GOLD_VENDAS_MENSAL ORDER BY MES;
SELECT COUNT(*) FROM OLIST_LAB.PUBLIC.SILVER_PEDIDOS; -- 99441 (a Silver tem 1 linha por pedido)
```

✓ Desafio Bronze concluído

SP lidera com ~R\$ 6 M em 41.746 pedidos; RJ tem 12,95% de atraso. Esses dados saíram do GitHub, viraram Bronze, Silver e Gold e passaram por 4 validações — sem ninguém tocar no Snowflake. Você usou Kubernetes sem perceber: cada task rodou num pod isolado.

Parte 2 — Duas experiências que valem mais do que parecem

Etapa 17 — Backfill: a task não sabe "hoje"

Você viu o professor fazer isso na aula. Agora é você. Abra a DAG `demo_backfill`. Ela tem 12 linhas e três diferenças em relação à principal: `schedule="@daily"`, `start_date` no passado, e a task lê `logical_date`.

1. Na página da DAG, clique na seta ao lado de "Acionar/Trigger" e escolha "Backfill".
2. Data inicial: 7 dias atrás. Data final: ontem. Marque a opção de NÃO reprocessar execuções existentes. Confirme.
3. Em segundos aparecem 7 execuções na Grid (7 colunas). Cada uma termina em menos de 1 minuto.
4. Abra o log de uma execução do meio: "Processando o dia 2026-xx-xx". Abra outra: outra data.

O que você acabou de provar

A mesma função Python rodou 7 vezes com 7 datas diferentes — e a data veio do orquestrador, não de `datetime.now()`. Um script que usa "agora" nunca consegue reprocessar o passado. Um pipeline que usa `logical_date` consegue. Essa é a diferença entre script e pipeline — e é o coração do desafio Prata B.

Etapa 18 — Idempotência: rodar duas vezes é seguro?

1. No Snowflake, anote o valor de `_ATUALIZADO_EM` da linha SP em `GOLD_RECEITA_ESTADO`.
2. No Airflow, acione `pipeline_olist_completo` de novo (Execução Única). Espere ficar verde (~2 min).
3. No Snowflake, rode o `SELECT` da Etapa 16 de novo.

Os números são idênticos; só `_ATUALIZADO_EM` mudou. Nenhuma linha duplicada, nenhuma receita dobrada. Por quê? A Bronze grava com `overwrite=True` e Silver/Gold usam `CREATE OR REPLACE`. Abra `dags/pipeline_olist.py` no Astro IDE e encontre essas três palavras.

Por que isso importa

Retry só é seguro em pipeline idempotente. Se a Bronze fizesse `INSERT` sem apagar, cada retry duplicaria 99 mil pedidos — e o dashboard mostraria o dobro da receita. Idempotência primeiro, retry depois. É o pilar nº 1 do checklist da aula.

Desafios (Extras)

Entrega: um PDF com os screenshots (seu nome de usuário visível) + o link do seu fork no GitHub.

Nível	O que entregar	Vale
 Bronze — obrigatório	Os 3 screenshots do Checkpoint 2 (Grid verde, log da Bronze com as 7 tabelas, SELECT na Gold)	até 7,0
 Prata — escolha 1 ou 2	Código no seu fork + evidência de execução (screenshots) + explicação de 5 linhas	+1,5 cada
 Ouro — bônus	DAG verde + explicação de 5 linhas do que mudou	+1,0 (teto 10)

Prata A — Faça um quality check falhar de verdade

Objetivo: entender que o quality check é um contrato e que a DAG protege o dashboard quando ele é violado.

1. No Astro IDE, abra dags/pipeline_olist.py e encontre a lista regras dentro de quality_checks.

```
("Nenhum estado com atraso > 10%", f"SELECT COUNT(*) FROM {gold} WHERE PCT_ATRASO > 10", lambda v: v == 0),
```

2. Commit + Deploy Project. Acione a DAG. quality_checks fica vermelha (3 tentativas — retry não resolve regra de negócio) e fim fica upstream_failed.
3. Abra o log de quality_checks: ele mostra FAIL na regra nova e o motivo.

Evidências

Screenshot da Grid com quality_checks vermelha e fim upstream_failed; screenshot do log com a linha FAIL; 5 linhas explicando: por que o fim não rodou, por que o retry não ajudou, e o que um time faria com esse alerta (corrigir o dado? relaxar a regra? bloquear o dashboard?).

Prata B — Dê um calendário à DAG e faça backfill

Objetivo: aplicar na DAG principal as três mudanças da demo_backfill.

1. Em dags/pipeline_olist.py: troque schedule=None por schedule="@daily" (start_date já está no passado; catchup=False continua).

```
def download_and_load_bronze(**context)) e imprimir context["logical_date"] na primeira linha do log.
```

2. Commit + Deploy. Trigger ▼ → Backfill → 3 dias (anteontem, ontem, hoje) → não reprocessar existentes.
3. Três execuções aparecem na Grid. Abra o log da Bronze de cada uma: três datas diferentes.

Pergunta para a explicação

As três execuções carregaram exatamente os mesmos dados. Por quê? O que precisaria mudar na Bronze para cada execução carregar SÓ os pedidos daquele dia? (Essa resposta é o desafio Ouro B.)

🏆 Prata C — Configure um alerta e prove que disparou

Objetivo: fechar o pilar "Alertas" que ficou com X no checklist da aula. Duas rotas:

Rota 1 — Astro Alerts (sem código). No Astro (não no Airflow): Deployment → Alerts → Add Alert → tipo "DAG failure" (ou "Task failure") → DAG pipeline_olist_completo → canal e-mail com o seu endereço. Salve. Provoque uma falha (use a regra do Prata A) e receba o e-mail.

Rota 2 — on_failure_callback (com código). Crie um webhook no Slack ou no Discord (Configurações do servidor → Integrações → Webhooks). Em dags/pipeline_olist.py, adicione uma função que faça `requests.post(WEBHOOK_URL, json={"content": f"Falhou: {context['task_instance'].task_id}"})` e registre em `default_args: "on_failure_callback":` notificar. Guarde a URL do webhook numa Variable do Airflow (Admin → Variables), nunca no código.

✅ Evidências

Screenshot do alerta configurado + screenshot do e-mail/mensagem recebido, com a task e o horário.

🏆 Ouro A — O projeto dbt da Aula 3 orquestrado pelo Airflow

A DAG pipeline_olist_dbt já está no repositório. Ela usa o Cosmos para transformar cada modelo dbt em uma task do Airflow (10 tasks, com run e test separados) e grava em schemas próprios: BRONZE, STAGING, SILVER, GOLD.

1. Se a sua Connection é por chave (Etapa 13B): Astro → Deployment → Environment → adicione SNOWFLAKE_AUTH = keypair. Se é por senha, não precisa fazer nada.
2. No Airflow, acione pipeline_olist_dbt. A primeira task carrega a Bronze em OLIST_LAB.BRONZE; o grupo dbt_transform roda os modelos; quality_checks valida OLIST_LAB.GOLD.FCT_RECEITA_ESTADO.
3. Abra a aba Graph: o lineage stg → int → fct aparece como tasks conectadas. Compare com o lineage que você viu no dbt Cloud na Aula 3.

✅ Evidências

Screenshot da Grid/Graph com as 10 tasks verdes; SELECT em OLIST_LAB.GOLD.FCT_RECEITA_ESTADO; 5 linhas comparando: o que muda entre transformar em Python (pipeline principal) e em dbt (esta DAG)? Onde ficam os testes em cada caso?

🏆 Ouro B — Bronze particionada por data (o pré-requisito real de backfill)

Objetivo: fazer cada execução carregar só os pedidos do seu logical_date — o que torna o backfill de verdade útil.

1. Na Bronze, depois de ler olist_orders_dataset.csv, filtre o DataFrame pela data de compra igual ao logical_date (o dataset vai de 2016 a 2018 — use `start_date=datetime(2017, 1, 1)` e escolha datas dessa faixa no backfill).
2. Troque `overwrite=True` por: `DELETE FROM BRONZE_ORDERS WHERE DATE(ORDER_PURCHASE_TIMESTAMP) = '<dia>'` seguido de `append` (`overwrite=False`). Isso é idempotência POR DATA.
3. Backfill de 3 dias de 2017. Confira no Snowflake: `SELECT DATE(ORDER_PURCHASE_TIMESTAMP), COUNT(*) FROM BRONZE_ORDERS GROUP BY 1` — só os 3 dias, sem duplicatas mesmo rodando duas vezes.

 Evidências

Trecho do código no fork; Grid com as 3 execuções; resultado do GROUP BY antes e depois de rodar o mesmo dia duas vezes (mesma contagem).

Troubleshooting — do erro à solução

Sintoma	Causa provável	Solução
DAG não aparece na lista após o deploy	Deployment ainda construindo a imagem	Espere HEALTHY (3–5 min) e recarregue. Persistiu? Veja "Import Errors" no topo da lista.
"Import Error" em pipeline_olist_dbt	Cosmos ou dbt não instalados na imagem	Confira requirements.txt (astronomer-cosmos==1.15.1) e o Dockerfile (venv dbt). Redeploy. Não afeta a DAG principal.
Bronze vermelha: "Multi-factor authentication", "MFA", "250001"	Snowflake exigindo MFA do usuário humano	Etapa 13B (usuário de serviço + chave).
Bronze vermelha: "Incorrect username or password"	Senha errada OU MFA (mesma mensagem)	Confira a senha no Snowflake pelo browser. Se entra lá e não entra aqui → MFA → Etapa 13B.
Bronze vermelha: "could not connect", "404", "account"	Account Identifier no formato errado	Extra JSON: "account": "ORG-CONTA", sem .snowflakecomputing.com, sem https.
Bronze vermelha: "No module named snowflake" / "pandas"	Imagem sem dependências	Etapa 10.3: Astro Project Path deve estar VAZIO. Reconecte e redeploy.
Bronze vermelha: "Insufficient privileges" / "does not exist"	Role sem permissão em OLIST_LAB	Com senha: role ACCOUNTADMIN no Extra. Com chave: rode o setup SQL inteiro (grants).
quality_checks vermelha, Bronze/Silver/Gold verdes	Warehouse suspenso no meio, ou regra do Prata A	Clique na task → Clear. Se foi o Prata A: comportamento esperado.
transform_silver: "Object BRONZE_ORDERS does not exist"	Bronze carregou em outro database/schema	Extra JSON: database OLIST_LAB. A DAG usa nomes qualificados OLIST_LAB.PUBLIC.*.
Deployment HIBERNATING	Fora do Wake Schedule	Deployment → Wake Schedule → ajuste. Volta em 2–3 min.
Task fica "queued" por minutos	Worker subindo (pod) ou Deployment acordando	Normal até ~1 min. Além disso: veja Deployment → Health.
Backfill não aparece no menu	DAG com schedule=None	Backfill exige schedule (Prata B).

Checklist de saída

- ☐ Deployment HEALTHY e Wake Schedule cobrindo o horário em que vou fazer os desafios
- ☐ pipeline_olist_completo com 6 tasks verdes (screenshot)
- ☐ Log de download_and_load_bronze com as 7 tabelas (screenshot)
- ☐ SELECT na GOLD_RECEITA_ESTADO com SP = 41746 pedidos (screenshot)
- ☐ Etapa 17: 7 execuções da demo_backfill com datas diferentes
- ☐ Etapa 18: rodei duas vezes e os números não mudaram
- ☐ Escolhi meu(s) desafio(s) Prata e sei que evidência entregar
- ☐ Sei onde está o log quando algo fica vermelho — e leio a última linha primeiro

O que você acabou de fazer

1. Criou um ambiente Airflow 3.2 gerenciado na nuvem (Astro), rodando em Kubernetes.
2. Conectou o seu fork do GitHub e fez deploy do código (Dockerfile + requirements.txt + DAGs em Python).
3. Configurou a conexão com o Snowflake com credenciais seguras — e, se precisou, do jeito que se faz em produção: usuário de serviço com chave.
4. Executou um pipeline que baixou 7 CSVs, criou 7 tabelas Bronze, 1 Silver com JOINS, 2 Gold com métricas de negócio e rodou 4 validações de qualidade.
5. Fez backfill de 7 dias e provou que a task usa a data do orquestrador, não "agora".
6. Rodou o pipeline duas vezes e provou que ele é idempotente.

Na vida real, a única diferença é: a fonte seria uma API ou um blob storage em vez do GitHub, o schedule seria diário em vez de manual, e o alerta iria para o Slack. O conceito é exatamente o mesmo — e você acabou de operá-lo.

Prof. Rafael S Novo Pereira — Data Integration e Pipelines — FIAP MBA Data Engineering